EV Powertrain Simulation: what we built and why

Explainer for review. Adaptive Control under Input/State Constraints (BIBS stability under sudden actuator saturation). Code, data and tests are in the project folder `ev-powertrain-sim`. Written October 27, 2026.

---

0. How to use this document

This is written so a reader who has not seen the code can follow every part, check the physics, and challenge the choices. Each section says **what the part does**, **why it was built that way**, **what was verified and how**, and **what is assumed or still wrong**. Section 11 lists questions worth putting to a reviewer. Section 10 lists the mistakes we found in our own work, because those are the most useful evidence of how far to trust the rest.

Reading rule. Anything marked **Verified** was measured or tested in the sandbox, and the number is quoted. Anything marked **Assumed** is a judgement, with no evidence behind it yet. Nothing in this report claims a stability proof. The Lyapunov decay bound is still open.

1. The project in plain words

An electric car's motor controller asks the inverter for a voltage. If the battery or inverter cannot deliver it, the controller still believes it did. A controller that *learns* a motor parameter online (an adaptive controller) can then mistake the shortfall for a wrong parameter, keep changing its estimate, and run away. The research question is how to guarantee the states stay bounded when that happens. The coding team's job is a simulation faithful enough to show the failure, test the fixes, and expose, as data, every place where what was **requested** differs from what was **delivered**.

The notes name the plant form `x' = f(x) + g(x)u + d(x,t)` and the saturation form `u = sat(u_c), Δu = u − u_c`. The whole simulation is organised so that each of those symbols is a column of data.

2. Architecture

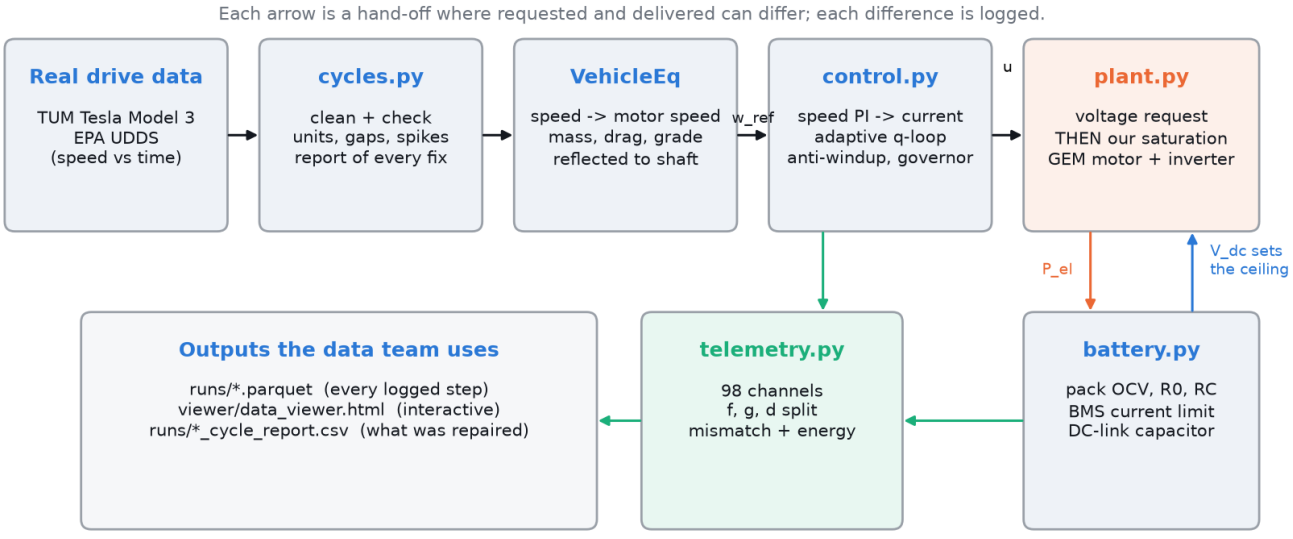


Figure 1. Data flows left to right along the top; energy flows between plant and battery; telemetry watches everything.

Part	File	Job in one sentence
Cycle loader	cycles.py	Turns any messy speed-vs-time file into a clean, checked speed request and records every repair.
Vehicle mapping	cycles.py: VehicleEq	Maps vehicle speed to motor speed and reflects mass, drag, rolling and grade onto the motor shaft.
Controller	control.py	Speed loop, adaptive current loop, anti-windup, and a governor that stops it asking for the impossible.
Plant adapter	plant.py	Wraps the Gym-Electric-Motor simulator; does the voltage saturation itself so it can be measured.
Battery + DC link	battery.py	Pack voltage, resistance, current limits and the bus capacitor; makes the voltage ceiling a state.
Telemetry	telemetry.py	Computes the f, g, d split, mismatch rows and energy books; holds the channel catalog.
Experiments	run_exp.py, run_cycle.py	Scenario runners for the 8 s experiments and for full real drives.
Viewer	viewer/	Self-contained page that plots every channel for any two runs.
Tests	tests/	60 automated checks; they pin the facts the project depends on.

3. The physics we are simulating

3.1 The motor (a permanent-magnet synchronous motor, PMSM)

A three-phase motor is easier to control after a change of coordinates (the dq frame) that rotates with the rotor. In that frame the motor is two coupled first-order circuits plus a mechanical equation:

```
L_d di_d/dt = u_d − R i_d + ω_el L_q i_q
L_q di_q/dt = u_q − R i_q − ω_el (L_d i_d + ψ)
T_elec      = 1.5 p (ψ + (L_d − L_q) i_d) i_q
J dω/dt     = T_elec − T_friction(ω) − T_disturbance(t)        ω_el = p ω
```

What each symbol means. *i_d, i_q* are currents along and across the magnet; *u_d, u_q* the voltages we command; *ψ* the magnet's flux linkage (0.066 Wb, the unknown the adaptive controller learns); *ω_el* the electrical speed; *R, L_d, L_q* the winding resistance and inductances (18 mΩ, 0.37 mH, 1.2 mH); *p = 3* pole pairs. The term *ω_el ψ* is the **back-EMF**: the voltage the spinning magnet generates against us. At 200 rad/s it is about 40 V.

Why torque is mostly i_q. With *i_d = 0* the torque is *1.5 p ψ i_q*, so the controller regulates *i_q* to make torque and holds *i_d* near zero. **Verified:** open-loop currents from the simulator match these equations to 1e-10 A, and torque matches the formula.

The speed limit of this simplification. The inverter can only produce a voltage of magnitude *V_dc/√3* (173 V at 300 V). At high speed the back-EMF and cross-coupling terms use up that budget, so holding *i_d = 0* becomes impossible above a *base speed* (about 195 rad/s at 240 A for this motor). Beyond it a real car injects negative *i_d* (field weakening). Tier 1 stays below that speed on purpose; field weakening is the hardest item in Tier 2.

3.2 The inverter and why saturation is a disc

The controller commands *(u_d, u_q)*. The inverter makes three phase voltages from three duty cycles in [−1, 1]. The simulator **clips duty cycles silently, leg by leg**, which makes the true limit a hexagon in the dq plane (direction dependent) and gives no record that clipping happened. **Verified** by probe scripts.

We therefore do the saturation ourselves, before the simulator sees the command: `u_app = u_cmd · min(1, u_lim/|u_cmd|)`, a radial projection onto a disc of radius *u_lim = V_dc/√3*. The disc is the largest

circle inside the hexagon, so it is isotropic, keeps the direction of the command, and is Lipschitz, which is what a proof wants. A zero-sequence offset (`duty −= 0.5(max+min)`) shifts all three legs equally, leaves the dq voltage unchanged, and lets the disc reach *V_dc/√3*. Because we pass only voltages inside the disc, the simulator never clips: the hidden-clipping check reads about 1e-11 V in every run.

4. plant.py: the adapter between our controllers and the simulator

Design rule: controllers see only this class, never the simulator. That is what lets Tier 2 replace the motor with a full vehicle without touching the controller.

- **Time-varying load.** The simulator's default load holds speed constant. We subclass its polynomial load (RoadLoad) so speed is a real state and an external torque *d(t)* can be injected. Parameters: constant (rolling) *a*, viscous *b*, quadratic (drag) *c*.
- **Constraints off.** The simulator ends an episode when current exceeds its limit. We pass `constraints=()` so a violation is logged as data, not a crash.
- **apply_voltage(u_cmd)** projects onto the ceiling, steps the simulator, and returns a ledger: requested and delivered voltages, *Δu*, a saturated flag, the ceiling, and a second figure *M3* (plant-reported minus ours) that must stay at zero.
- **Battery hook.** If a battery is supplied, the ceiling each step is *V_dc/√3* from the battery state, and the motor's electrical power is fed back to it. The simulator's own supply stays a fixed 300 V numerical scale (the duty cycle is computed against it) because we always command the voltage we want; only our ceiling changes.
- **New constructor arguments:** step size *tau* (1e-4 s default, 2e-4 s for cycle runs) and the load coefficients.

Known limitation. When regeneration pushes the real bus above 300 V, the true ceiling would be higher, but the simulator's 300 V scale would clip. We cap the ceiling at 173.2 V to keep hidden clipping at zero, which wastes up to about 4 V of headroom during regen. This bug was found by our own check (section 10).

5. control.py: the controller, and why each piece exists

Structure. A cascade, as in a real drive: a slow speed loop produces a current request; fast current loops produce voltage requests.

```
speed PI (every 10 steps) ──> i_q* (clipped to i_lim = 250 A, or to the governor limit)
q-axis ADAPTIVE current loop ──> u_q,c            d-axis certainty-equivalence loop ──> u_d,c
```

5.1 The adaptive q-axis loop (the heart of the project)

The unknown is the flux linkage *ψ*. Define the tracking error $e = i_q - i_q^*$ and the estimate error $\tilde{\psi} = \hat{\psi} - \psi$. The control law cancels what it knows and uses the estimate for the rest:

```
u_q,c = R i_q + ω_el (L_d i_d + ψ^) − k e
Error dynamics (no saturation):   L_q de/dt = −k e + ω_el ψ~
Lyapunov function:                V = ½ L_q e² + ψ~² / (2Γ)
V-dot = −k e² + ψ~ (ω_el e + dψ^/dt / Γ)   ⇒   choose   dψ^/dt = −Γ ω_el e     (V-dot = −k e² ≤ 0)
```

Why the sign matters. An earlier draft had the sign of the *ω_el ψ~* term wrong and the estimate diverged to −6.8 Wb even without saturation. This is the single most likely place for a future re-derivation to go wrong, so the code carries a warning. **Verified** by simulation: with no saturation the estimate converges and stays within 0.0198 Wb of truth.

Why the unknown multiplies speed. The regressor is *ω_el*, so the estimator learns fastest exactly where the voltage ceiling is tightest (high speed).

5.2 How saturation breaks it, and the three fixes

When the actuator saturates, the true error dynamics gain a term: $L_q\, de/dt = -k\, e + \omega_{el}\, \psi\sim + \Delta u_q$. The adaptive law cannot tell Δu_q from $\psi\sim$, so it moves the estimate to explain an error that has nothing to do with ψ. That is adaptive windup.

Mode	Idea	What it changes
naive (B)	no protection	$d\hat{\psi}/dt = -\Gamma\, \omega_{el}\, e$. The estimate is fooled by Δu_q.
robust (C)	σ-modification + projection	Adds a leak toward the nominal value, $-\sigma(\hat{\psi} - \psi_{nom})$, and clips $\hat{\psi}$ to [0.03, 0.10] Wb. We leak toward nominal, not toward zero, because ψ is far from zero (the notes write the zero version; Theory should confirm the change).
aw (D)	anti-windup auxiliary state	$L_q\, dz/dt = -k\, z + \Delta u_q$, and the adaptive law uses $e_a = e - z$. For this scalar loop z is the part of the error caused by the actuator, so e_a obeys the unsaturated error dynamics exactly and the estimate is not fooled.

5.3 The governor (reference governor in miniature)

Even with a protected estimator, asking for current the voltage cannot sustain leaves the drive in a trap (we measured 230 A producing only 18 N·m, because the d-axis cross-coupling voltage consumed the budget). The steady-state voltage ellipse gives the largest i_q the ceiling can sustain at a speed: $(L_q\, i_q)^2 + \psi^2 \le (u_{max}/\omega_{el})^2$. The governor clips the current request to a safety factor (0.75) of that. With `vdc_aware=True` it uses the *measured* bus voltage instead of the nominal one. That single change is why exposing V_{dc} as a channel matters.

6. battery.py: the battery pack and DC link

Why it exists. The notes list DC-link voltage and battery state of charge among the states. More importantly, the saturation event is far more convincing when the powertrain produces it than when we impose a number.

```
battery  (OCV(SOC), R0(T), one RC branch, BMS current limits)
   │  busbar resistance Rc
   ▼
DC-link capacitor C (voltage state V_dc)  ⟶  inverter  ⟶  motor
```

- **Pack.** 80 cells in series, 60 Ah. Open-circuit voltage from a generic NMC curve (302.4 V at 50 %). Ohmic resistance 0.08 Ω at 25 °C rising when cold. One RC branch (0.048 Ω, 20 s) for diffusion. Heating and cooling are modelled, but temperature barely moves in 8 s.
- **Bus.** $C\, dV_{dc}/dt = i_{batt} - i_{dc}$ with $i_{dc} = P_{dc} / V_{dc}$ and $P_{dc} = P_{el} + \textit{inverter loss}$ (idle 30 W plus a conduction term). $C = 2$ *mF*.
- **BMS limit.** Pack current is clamped to +250 A / −150 A. When the limit binds, the capacitor supplies the shortfall, and the bus falls in milliseconds. The voltage the limiter drops is dissipated there (P_{bms_clip}), which keeps the energy books closed.
- **Numerics.** The capacitor-resistor loop has a time constant of about 0.3 ms, comparable to the 0.1 ms step, so explicit Euler would be marginal. The bus update is solved implicitly (one linear equation) and the RC branch with its exact exponential.

Verified. Resting bus equals the open-circuit voltage; a 10 kW draw gives the expected resistive drop; regeneration charges the pack; the pack's energy chain (chemical power − losses − bus power − limiter heat) closes to 1e-11 W at every step; the bus current balance closes to 0.07 W rms.

Assumed, not fitted. Every pack number is a generic lithium-ion value. In section 8 a real Tesla trace suggests the effective pack resistance is about 0.062 Ω against our 0.085 Ω, which is a calibration task, not a verified fact.

7. telemetry.py: the data product

The plant logs what it measures. `derive()` adds everything an analyst needs, as pure functions of logged columns, so the same code applies to any run or to a future vehicle plant. `CATALOG` lists 98 channels with unit, group, role and meaning.

7.1 The f, g, d split

```
di_d/dt = (−R i_d + ω_el L_q i_q)/L_d  +  u_d,cmd/L_d  +  Δu_d/L_d       f_id,  g_id u,  d_id
di_q/dt = (−R i_q − ω_el(L_d i_d+ψ))/L_q + u_q,cmd/L_q + Δu_q/L_q       f_iq,  g_iq u,  d_iq
dω/dt   = (T_elec − T_fric)/J                − T_dist/J                    f_w,           d_w
```

The actuator disturbance *d_id, d_iq* is literally $g \cdot \Delta u$: the notes' *d(u,x)*, a disturbance that depends on the input. The road load *d_w* is the external disturbance. **Verified:** adding these terms reproduces the measured derivatives within 0.03 to 0.85 % rms (varies by run), which is the check that the columns mean what they say.

7.2 The mismatch ledger

Sign convention everywhere: **delta = delivered − requested**. Negative means the stage gave less than it was asked.

Row	Between	Channel(s)	Meaning
M1	driver → controller	e_speed	Speed request minus actual speed.
M2	controller → inverter	d_u_d, d_u_q, d_u_norm, saturated	Voltage delivered minus requested, and a flag.
M3	our saturation → simulator	d_u_plant_d/q	Must be zero: proves no hidden clipping.
M4	speed loop → current	i_ref_clip	Current request after clipping minus before.
M5	current → shaft	d_T	Actual torque minus the torque the current request implied.
M6	electrical power	dP_elec, dP_mech	Delivered minus requested power. Use the shaft version (see caution).
M8	energy	E_mech_unserved	Cumulative shaft energy requested and not delivered.
M9	battery → bus	bus_sag, ceiling_loss	How far the bus fell below open-circuit, and the inverter headroom it cost.
M10	battery → inverter	d_i_dc, E_bus_unserved	Pack current delivered minus inverter current requested.

Caution found by testing. The electrical "requested power" $u_{cmd} \cdot i$ mixes real power with reactive cross-coupling voltage, and the current responds to what was *delivered*, so it is a counterfactual and can be negative. The shaft-power version ($T_{cmd}\,\omega$ against $T_{elec}\,\omega$) is unambiguous and is the one to use. **Also:** a governor that lowers the current request makes M4/M5 look small while the speed shortfall (M1) is large. Mismatch depends on where you measure, so M1 is the complete number.

8. Real data: the TUM Tesla Model 3 dataset and cycles.py

8.1 The dataset

Electric powertrain data of a Tesla Model 3 during real-world driving situations (Rosenberger et al., TUM, 2024; CC BY 4.0; doi:10.14459/2024mp1735471; paper doi:10.3390/wevj15060268). 228 measurements recorded from the vehicle's CAN bus: high-voltage battery (current, voltage, temperatures, state of charge), powertrain (rotation, torque, temperature) and vehicle (speed, mileage). 6.3 GB as CSV and MAT files.

What we actually did. The share allows listing and per-file download. We downloaded five tracks (0, 1, 10, 100, 103; 1.3 to 10.6 MB), verified each against the published SHA-512 checksums (all five match), and ran them. We did not download the other 223.

8.2 What the real data taught us (all measured on the files)

- **Time** is epoch milliseconds (values near 1.6e12), about 10 Hz.
- **Schemas differ between tracks.** Tracks 10, 100 and 103 have no `engine_rpm` column; every optional channel is looked up by name.
- **Column names mislead.** `engine_rpm` is the *wheel* rpm: speed divided by it implies a 0.331 m radius, which matches a Model 3 tyre (a motor-rpm reading implies 3 m, which is absurd).
- **Battery current sign.** Discharge is negative: current correlates −0.86 with vehicle kinetic power.
- **rear_motor_torque is not trusted.** It is non-negative and correlates −0.46 with kinetic power, so its meaning is unclear. We do not use it.
- **Speed has gaps.** Track 1's GPS speed has 885 missing samples (4.9 %), interpolated and reported; 10 isolated spikes removed.
- **GPS elevation glitches.** Naive differentiation gave a 24.5 % grade; a median filter, a 30 s window and a ±15 % cap give 0.5° to 5.6°.

8.3 cycles.py: handling bad data on purpose

Every anomaly is handled one of three ways and recorded in a report that is saved next to each run: **FIX** (repaired and noted), **ERROR** (refused with a named error and remedy), or **WARN** (accepted but suspicious). Nothing is repaired silently. 51 automated cases cover the list below; writing them found six bugs in our own loader (section 10).

Group	Cases covered (each is a test)
File	missing; directory; empty; only comments; UTF-8 BOM; Latin-1 bytes; ragged rows; title lines before the header; semicolon, tab, pipe and whitespace delimiters; decimal commas; 400,000-row file under 15 s
Columns and units	ambiguous or missing columns; mph, km/h and m/s from header or name; unitless values guessed and flagged; unknown unit refused; implausible speed (wrong units) refused; text such as "12.3 m/s" or "n/a" coerced
Time	epoch milliseconds; explicit time unit; out of order; duplicate timestamps; large gaps (keep longest stretch, or refuse); too short; irregular sampling resampled to a uniform grid; all-missing time
Speed	missing samples interpolated; mostly missing refused; all missing refused; small negatives clipped; reversing refused; isolated spikes removed; infinite values; constant speed and standstill warned
Policy	strict mode turns warnings into errors; arrays accepted; identical input gives identical output; extra measured channels aligned to the grid
Vehicle fit	gentle cycle fits; aggressive cycle refused with a remedy; motionless cycle refused; synthetic TUM schema variants (optional channels absent, GPS speed missing, no time column); five real tracks sane

8.4 Putting a real vehicle on a small motor

The Tier 1 motor makes at most about 74 N·m ($1.5\,p\,\psi \times 250$ A with $i_d = 0$) and should stay below about 195 rad/s. A real car needs far more of both. `fit_vehicle_to_motor` picks a gear ratio so the cycle's peak speed maps to 190 rad/s, then the largest **equivalent mass** whose torque demand stays under 85 % of capability for 99 % of the cycle. Road load (rolling, drag) and grade are reflected to the shaft as $m\,g\,C_{rr}/k$, $\tfrac{1}{2}\rho\,C_dA/k^3$ and $m\,g\,\sin\theta/k$, and inertia as m/k^2, where k is motor speed per vehicle speed.

Track	Length	Peak speed	Fits the motor?	Equivalent mass
track_0	283 s	23.8 m/s	Yes	130 kg

Track	Length	Peak speed	Fits the motor?	Equivalent mass
track_10	841 s	28.2 m/s	Yes	102 kg
track_103	365 s	22.8 m/s	Yes	118 kg
track_1	1,796 s	38.4 m/s	No: drag exceeds the motor; fits only with road load scaled to 59 %	20 kg (minimum)
track_100	293 s	39.0 m/s	No: fits only with road load scaled to 51 %	20 kg (minimum)

Refusing the two highway tracks is the correct behaviour, not a failure: it is the same field-weakening limit that makes Tier 2 hard. The loader says so and offers an explicit, recorded scaling option.

Assumed. The equivalent mass is a toy number, not the Tesla's 1,900 kg. The drag area (0.62 m²), rolling coefficient (0.010) and the 190 rad/s target are assumptions. EPA UDDS also loads cleanly (1,369 s, 25.3 m/s, fits at 287 kg) but has not yet been run through the powertrain.

9. What the experiments show

9.1 The 8 s scenarios on the plain motor

Speed ramps to 200 rad/s, a 10 N·m grade load starts at 1.5 s, and at 4.5 s the voltage ceiling drops from 173 V to 25 V for one second (an unannounced event). Identical across runs so only the controller differs.

Run	Controller	Estimate ψ^ peak (true 0.066)	Peak current	Speed error rms after event	Peak Δu
A	naive, no event	0.066	151 A	0.0 rad/s	0 V
B	naive	2.741 Wb (runaway)	589 A	64.1 rad/s	2,068 V
C	σ-mod + projection	0.100 (bounded)	255 A	6.0 rad/s	825 V
D	anti-windup	0.0665	250 A	0.81 rad/s	806 V
Dg	anti-windup + governor	0.0662	224 A	0.07 rad/s	706 V

This reproduces the failure the notes predicted (B: the estimate runs away to 41 times its true value) and shows both proposed fixes work, with anti-windup better than the leak.

9.2 With the battery: the event is now physical

The BMS cuts the allowed pack current to 5 A for one second at 4.5 s, below the 12.6 A cruise demand. The bus collapses to about 60 V. Unprotected: speed dips to 137 rad/s, reaches only 184 by the end, peak voltage shortfall 924 V, 35.7 kJ of shaft energy owed. Protected (anti-windup, governor, sees the bus voltage): dips to 172, recovers to 200, peak shortfall 231 V, 0.1 kJ owed.

9.3 A real drive

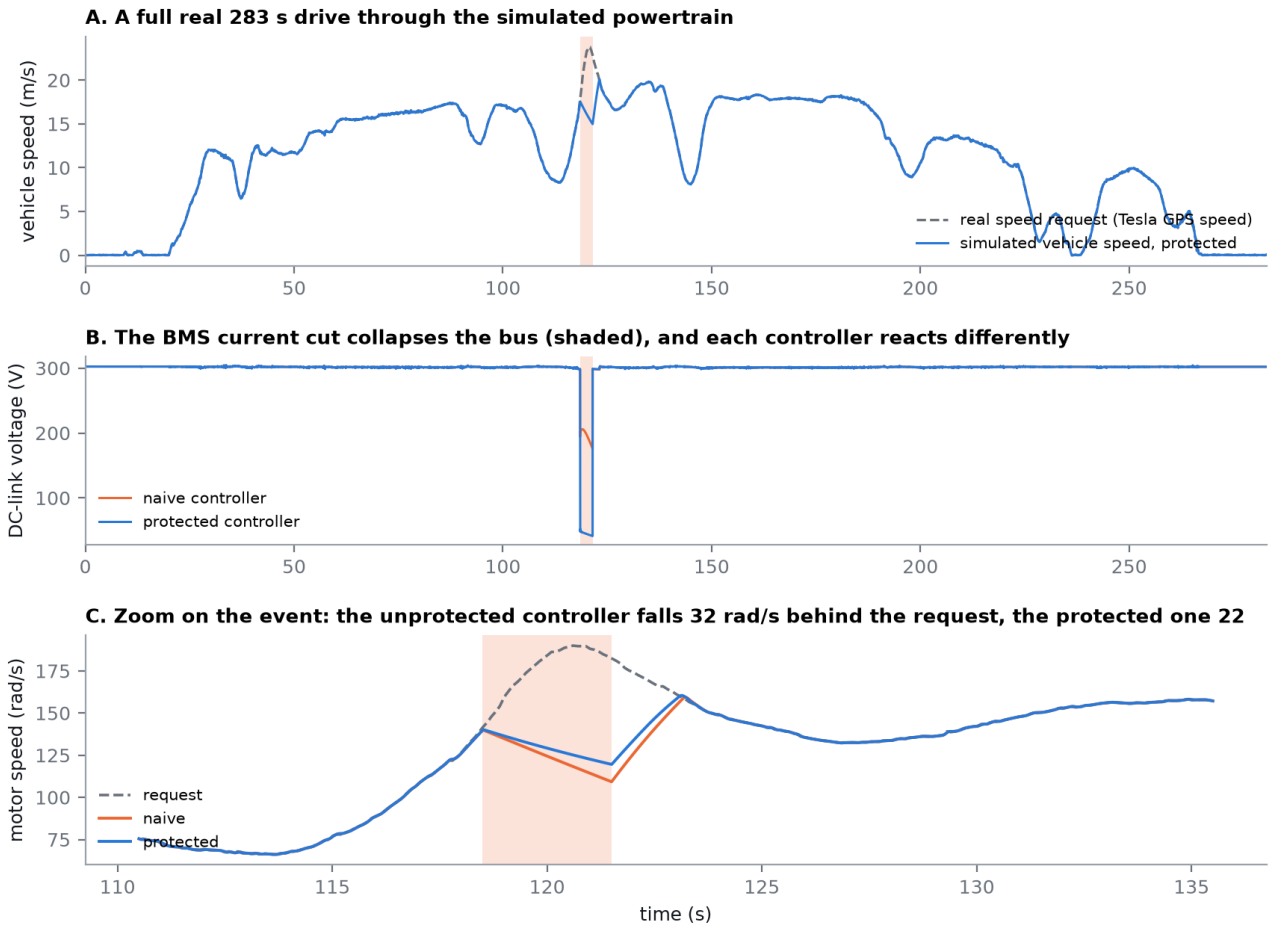


Figure 2. TUM track_0 (a real Tesla Model 3 drive) driving the simulated powertrain. The BMS cut is applied for 3 s at the moment of peak acceleration power (118.5 s).

Verified: both 283 s runs completed (56,580 logged rows each, 1.4 million physics steps, about 400 s of compute), nothing diverged, every channel is finite. Away from the event the simulated speed follows the real one closely (tracking error 3 to 3.5 % of peak). During the event the unprotected controller falls 32 rad/s behind the request and the protected one 22; the protected run's bus voltage falls further (41 V against 176 V) because the controller is asking for less, and it still tracks better afterwards.

9.4 How well does the simulated battery resemble the real one?

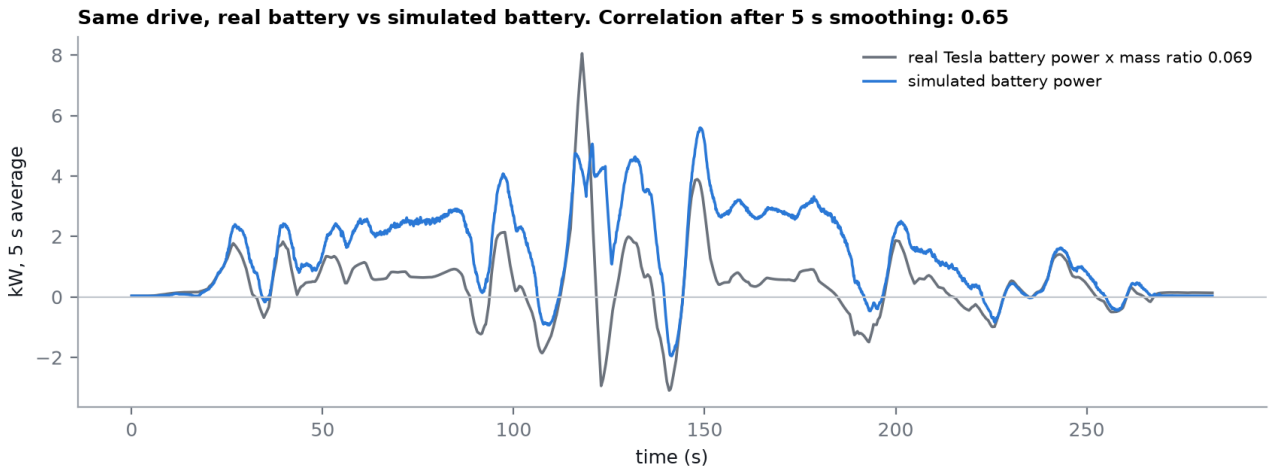


Figure 3. Battery power during the same drive. The real trace is scaled by the mass ratio 0.069 (equivalent mass over 1,900 kg).

- **Agreement is moderate:** correlation 0.40 at 0.1 s, 0.52 at 1 s, 0.65 at 5 s smoothing.
- **Regeneration disagrees:** real 38 % of drive energy, simulated 12 %. The toy vehicle and the simple deceleration behaviour are likely causes; not yet investigated.
- **Net energy disagrees:** 0.035 kWh (real, scaled) against 0.116 kWh simulated.
- **Pack resistance:** a fit to the real data gives about 0.062 Ω against our 0.085 Ω.
- **What this does and does not show.** It shows the pipeline runs end to end on real data and gives a first calibration target. It does not show the simulated powertrain predicts a real Tesla; the vehicle is a scaled toy.

10. Mistakes we found in our own work

These are listed because they show what the checks catch and where trust should be limited.

#	What went wrong	How it was found	Fix
1	Adaptive law had the wrong sign; the estimate diverged to −6.8 Wb with no saturation	Baseline run	Re-derived; warning left in the code
2	Scenario at 300 rad/s was infeasible (cross-coupling 216 V > 173 V)	Voltage ellipse probe	Lowered to 200 rad/s
3	After the event the drive sat in a trap (230 A, 18 N·m)	Experiments C and D	Added the governor
4	np.cross on 2-vectors fails in NumPy 2	Unit test	Explicit 2-D cross product
5	Simulator clipped silently up to 4 V when regeneration pushed the bus above 300 V	The viewer's own M3 check showed 3.7 V	Capped the ceiling; regression test
6	Electrical "requested power" came out negative	Cumulative energy looked wrong	Added the shaft-power mismatch; caveat documented
7	A 25 A BMS derate never bound (cruise draws 12.6 A), so no event occurred	Run summary showed no sag	Derate set to 5 A
8	utf-16 decoding accepted arbitrary bytes as garbage text	Test 06	utf-16 only with a byte-order mark
9	Unit names like speed_mph were not recognised (underscore is a word character)	Test 13	Lookaround regular expressions
10	Read-only arrays crashed spike handling	Test 34	Copy on load

#	What went wrong	How it was found	Fix
11	A dirty data row ("n/a") was taken as the header	Test 18	Header = nearest all-text row
12	Time unit inferred from unsorted time (a 0.3 s cycle)	Test 21	Infer from sorted differences
13	Naive GPS elevation differentiation gave a 24.5 % grade	Real track 1	Median filter, 30 s window, cap
14	Full-rate validation checks showed misleading "pass" or large errors on decimated real runs	Viewer on the real drive	Those checks now show SKIP at low logging rates

11. Assumptions, limits and questions for a reviewer

11.1 Not yet true (limits)

- **No stability proof.** V-dot ≤ −λV + c is unverified. The logged V(t) is a candidate only.
- **One unknown parameter (ψ) and one kind of event** per run. No noise, no sensor delay, no parameter drift yet.
- **Tier 1 voltage ceiling is capped at 173.2 V**; real field weakening is not implemented.
- **The vehicle is a toy:** equivalent mass, assumed drag and rolling coefficients, assumed gear ratio.
- **Battery numbers are generic** (OCV table, resistances, BMS limits, inverter loss, thermal mass). A real pack would differ.
- **Five of 228 real tracks** were used. No other dataset (for example Argonne D3, which blocked automated download) has been opened.
- **Full-rate checks need full-rate logs.** The 283 s runs were logged at 200 Hz, so those validation rows are skipped.
- **Sensor noise and a cycle-dataset sweep are the next build**, not done.

11.2 Good questions to put to a reviewer

- Is the sign and form of $d\psi\hat{}/dt = -\Gamma\ \omega_el\ e_a$ correct, and does the compensated error $e_a = e - z$ really recover the unsaturated dynamics for this loop?
- Is leaking toward the *nominal* value (not zero) a legitimate σ-modification for a parameter far from zero, and does it keep the proof's inequality?
- Is a radial projection onto the disc of radius $V_dc/\sqrt{3}$ an acceptable saturation model for a BIBS or UUB theorem? What does it assume about the actuator?
- Does the implicit bus solve with an explicit constant-power current (using the previous V_dc) risk a spurious oscillation near collapse?
- The governor lowers the current request, which hides the shortfall at M4 and M5. Which mismatch should the proof be stated in terms of?
- Is the equivalent-mass scaling a defensible way to run a real vehicle trace on a smaller motor, or should we scale power instead?
- What would a fair calibration of the pack resistance from the real trace look like, given the vehicle is scaled?

12. Glossary

Term	Meaning here
PMSM	Permanent-magnet synchronous motor: the motor type used in most EVs.
dq frame	Coordinates rotating with the rotor; turns three phase quantities into two DC-like ones.
Back-EMF	Voltage the spinning magnet generates; grows with speed (ω_el ψ).
Field weakening	Injecting negative i_d to reduce flux so the motor can spin faster than the voltage budget otherwise allows.
Inverter / duty cycle	Power electronics that turn DC bus voltage into three phase voltages; duty cycle is each leg's on-time fraction.
DC link / bus	The capacitor-smoothed DC voltage between battery and inverter.
BMS	Battery management system; limits pack current and voltage.
OCV, SOC	Open-circuit voltage and state of charge of the pack.
Adaptive control	A controller that estimates an unknown parameter online.
Windup	An integrator or estimator accumulating error the actuator cannot act on.
σ-modification, projection	Leakage that pulls an estimate back, and a hard clamp that keeps it in a physical range.
Anti-windup	Extra state that tracks the actuator's shortfall so the controller can discount it.
Reference governor	A filter that stops the reference asking for something the system cannot do.
Lyapunov function V	An energy-like function; if it cannot grow, the state stays bounded.
BIBS, UUB	Bounded-input bounded-state stability; uniform ultimate boundedness (states end inside a calculable bound).
Equivalent mass	The vehicle mass, reflected to the motor, that the toy motor can actually accelerate.

13. Sources

- Rosenberger N. et al. (2024). Electric powertrain data of a Tesla Model 3 during real-world driving situations. TUM mediaTUM. https://doi.org/10.14459/2024mp1735471 (CC BY 4.0). Related paper: https://doi.org/10.3390/wevj15060268
- U.S. EPA Urban Dynamometer Driving Schedule (UDDS) speed trace, EPA dynamometer drive schedules.
- Gym-Electric-Motor 3.0.3 (Cont-CC-PMSM-v0) and FASTSim 3.1.0 open-source packages.
- Pozzato G. et al. Real-world electric vehicle data: driving and charging. Mendeley Data. https://doi.org/10.17632/7vdkzpnjgj.2 (identified as a candidate; its files were not downloaded).